DS Practical Solutions

1. Delete the smallest node in a Binary Search Tree (Implementation Level)

```
void deleteSmallestNode(Tree *t){
    if (*t == NULL) return;
    NodeT *pre = NULL;
    NodeT *cur = *t;
    while(cur->left != NULL){
        pre = cur;
        cur = cur->left;
    }
    if(pre == NULL)
        *t = cur->right;
    else
        pre->left = cur->right;
    free(cur);
}
```

---

2. Delete the leaves node from a Tree (Implementation Level)

```
void deleteLeavesNode(Tree *t){
    if(*t == NULL) return;

    if((*t)->left != NULL && (*t)->left->left == NULL && (*t)->left->right ==
NULL){
        NodeT *temp = (*t)->left;
        (*t)->left = NULL;
        free(temp);
    } else {
        deleteLeavesNode(&((*t)->left));
    }

    if((*t)->right != NULL && (*t)->right->left == NULL && (*t)->right->right
== NULL){
        NodeT *temp = (*t)->right;
        (*t)->right = NULL;
        free(temp);
    } else {
        deleteLeavesNode(&((*t)->right));
    }
}
```

---

3. Search in a Queue (Linked List Based) and push it to Stack

```
void searchqq(Queuel *q, Type key, Stack *s){
    if(isEmptyQueue(q)){
        printf("Empty\n");
        return;
    }
    Nodeq *cur = q->head;
    while(cur != NULL){
        if(cur->info == key){
            printf("Found\n");
            push(s, key);
            return;
        }
        cur = cur->next;
    }
    printf("Key not found\n");
}
```

---

4. Search in a Queue (Array Based) and push it to Stack

```
void searchq(Queue *q, Type key, Stack *s) {
    if (isEmpty(*q)) {
        printf("Empty\n");
        return;
    }
    for (int i = 0; i < q->size; i++) {
        int j = (q->front + 1) % MAX;
        if (q->arr[j] == key) {
            printf("Found\n");
            push(s, key);
            return;
        }
    }
    printf("Not Found\n");
}
```

---

5. Delete the largest node from a Binary Search Tree

```
void deleteLargestNode(Tree *t){
    if (*t == NULL) return;
    NodeT *pre = NULL;
    NodeT *cur = *t;
    while(cur->right != NULL){
        pre = cur;
        cur = cur->right;
    }
    if(pre == NULL)
        *t = cur->left;
    else
        pre->right = cur->left;
    free(cur);}
```

---

6. Count the number of nodes in a Binary Tree

```
int countNodes(NodeT *root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

---

7. Find the height of a Binary Tree

```
int findHeight(NodeT *root) {
    if (root == NULL) return -1;
    int lh = findHeight(root->left);
    int rh = findHeight(root->right);
    return 1 + (lh > rh ? lh : rh);
}
```

---

8. Mirror a Binary Tree

```
void mirrorTree(NodeT *root) {
    if (root == NULL) return;
    NodeT *temp = root->left;
    root->left = root->right;
    root->right = temp;
    mirrorTree(root->left);
    mirrorTree(root->right);
}
```

---

9. Print Inorder Traversal Without Recursion

```
void inorder(NodeT *root) {
    Stack s;
    initStack(&s);
    NodeT *curr = root;
    while (curr != NULL || !isEmpty(&s)) {
        while (curr != NULL) {
            push(&s, curr);
            curr = curr->left;
        }
        curr = pop(&s);
        printf("%d ", curr->info);
        curr = curr->right;
    }
}
```

---

10. Reverse a Queue using Stack

```
void reverseQueue(Queue *q, Stack *s){
    while (!isEmptyQueue(q)) {
        push(s, dequeue(q));
    }
    while (!isEmpty(s)) {
        enqueue(q, pop(s));
    }
}
```

---

11. Copy all elements of one queue to another

```
void copyQueue(Queue *src, Queue *dest){
    NodeQ *cur = src->head;
    while(cur != NULL){
        enqueue(dest, cur->info);
        cur = cur->next;
    }
}
```

---

12. Check if Parentheses are Balanced

```
int isBalanced(char *expr) {
    Stack s;
    initStack(&s);
    for (int i = 0; expr[i] != '\0'; i++) {
        if (expr[i] == '(') push(&s, '(');
        else if (expr[i] == ')') {
            if (isEmpty(&s)) return 0;
            pop(&s);
        }
    }
    return isEmpty(&s);
}
```

---

13. Reverse a String using Stack

```
void reverseString(char *str) {
    Stack s;
    initStack(&s);
    for (int i = 0; str[i] != '\0'; i++) push(&s, str[i]);
    for (int i = 0; !isEmpty(&s); i++) str[i] = pop(&s);
}
```

---

14. Sort a Stack using Another Stack

```
void sortStack(Stack *s) {
    Stack temp;
    initStack(&temp);
    while (!isEmpty(s)) {
        int tmp = pop(s);
        while (!isEmpty(&temp) && peek(&temp) > tmp) {
            push(s, pop(&temp));
        }
        push(&temp, tmp);
    }
    *s = temp;
}
```

---

15. Insert in Sorted Linked List

```
void insertSorted(Node **head, int data){
    Node *newNode = createNode(data);
    if (*head == NULL || (*head)->data >= data){
        newNode->next = *head;
        *head = newNode;
        return;
    }
    Node *cur = *head;
    while(cur->next != NULL && cur->next->data < data)
        cur = cur->next;
    newNode->next = cur->next;
    cur->next = newNode;
}
```

---

16. Delete All Even Nodes in Linked List

```
void deleteEven(Node **head){
    Node *cur = *head, *prev = NULL;
    while(cur != NULL){
        if(cur->data % 2 == 0){
            Node *temp = cur;
            if(prev == NULL)
                *head = cur->next;
            else
                prev->next = cur->next;
            cur = cur->next;
            free(temp);
        }else{
            prev = cur;
            cur = cur->next;
```

```
        }
    }
}
```

---

17. Merge Two Sorted Linked Lists

```
Node* mergeSorted(Node *a, Node *b){
    if (!a) return b;
    if (!b) return a;
    if (a->data < b->data){
        a->next = mergeSorted(a->next, b);
        return a;
    }else{
        b->next = mergeSorted(a, b->next);
        return b;
    }
}
```

---

18. Detect Loop in Linked List

```
int detectLoop(Node *head){
    Node *slow = head, *fast = head;
    while (fast && fast->next){
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1;
    }
    return 0;
}
```

---

19. Reverse a Linked List

```
Node* reverse(Node *head){
    Node *prev = NULL, *cur = head, *next = NULL;
    while (cur != NULL){
        next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    return prev;
}
```

---

20. Implement Enqueue in Circular Queue

```
void enqueue(CQueue *q, int value) {
    if ((q->rear + 1) % MAX == q->front) {
        printf("Queue Full\n");
```

```
        return;
    }
    q->rear = (q->rear + 1) % MAX;
    q->arr[q->rear] = value;
}
```

---

21. Implement Dequeue in Circular Queue

```
int dequeue(CQueue *q) {
    if (q->front == q->rear) {
        printf("Queue Empty\n");
        return -1;
    }
    q->front = (q->front + 1) % MAX;
    return q->arr[q->front];
}
```

---

22. Count Elements in a Queue

```
int countQueue(Queue *q){
    int count = 0;
    NodeQ *cur = q->head;
    while(cur != NULL){
        count++;
        cur = cur->next;
    }
    return count;
}
```

---

23. Find Max Value in Linked List

```
int findMax(Node *head){
    if (head == NULL) return -1;
    int max = head->data;
    Node *cur = head->next;
    while(cur != NULL){
        if(cur->data > max)
            max = cur->data;
        cur = cur->next;
    }
    return max;
}
```

---

24. Find Minimum in Binary Tree

```
int findMin(NodeT *root){
    if (root == NULL) return INT_MAX;
    int l = findMin(root->left);
```

```
    int r = findMin(root->right);
    return min(root->info, min(l, r));
}
```

---

25. Count the number of leaf nodes in a Binary Tree

```
int countLeaves(NodeT *root){
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}
```
